

MSCIT SEM 1 - MONGODB

Chapter 4

Aggregation Queries

- (1) Generating sample data**
- (2) Understanding Map Reduce.**
- (3) Performing Map Reduce in Monod and PHP, Aggregation using group()**
- (4) Listing distinct values for field.**
- (5) Counting documents with count()**

Explain aggregation in Mongodb.

Ans: -

Once you have data stored in Mongodb then you may want to do more than just data retrieval.

You may want to analysis or display the data in any interesting way at that time Mongodb provides the aggregation framework, aggregation map command, map reduce support etc.

Aggregation operations process the data record and return computed results.

The aggregation framework transfer and combined the documents in a collection. It means that aggregation operation is one-group values from multiple document together and return one single result.

Shree Matrumandir College Of IT & MGMT

In SQL count (*) and group by is equivalent to MongoDB aggregation.

The aggregate () is used for aggregation in MongoDB.

Syntax: - db.collectionname.aggregate(pipeline, operations)

Example: - db.stud.aggregate([{\$group:{_id:'\$gender' total: {\$sum:1}}}}])

Explain aggregation expressions.

Ans:

Expression	Description	Example
\$sum	Sums up the defined value from all documents in the collection.	db.mycol.aggregate([{\$group : { _id : "\$by_user", num_tutorial : {\$sum : "\$likes"}}}])
\$avg	Calculates the average of all given values from all documents in the collection.	db.mycol.aggregate([{\$group : { _id : "\$by_user", num_tutorial : {\$avg : "\$likes"}}}])
\$min	Gets the minimum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{\$group : { _id : "\$by_user", num_tutorial : {\$min : "\$likes"}}}])
\$max	Gets the maximum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{\$group : { _id : "\$by_user", num_tutorial : {\$max : "\$likes"}}}])
\$push	Inserts the value to an array in the resulting document.	db.mycol.aggregate([{\$group : { _id : "\$by_user", url : {\$push : "\$url"}}}])
\$addToSet	Inserts the value to an array in the resulting document but does not create duplicates.	db.mycol.aggregate([{\$group : { _id : "\$by_user", url : {\$addToSet : "\$url"}}}])
\$first	Gets the first document from the source documents according to the grouping. Typically this makes only sense together with some previously applied "\$sort"-stage.	db.mycol.aggregate([{\$group : { _id : "\$by_user", first_url : {\$first : "\$url"}}}])
\$last	Gets the last document from the source documents according to the grouping. Typically this makes only sense together with some previously applied "\$sort"-stage.	db.mycol.aggregate([{\$group : { _id : "\$by_user", last_url : {\$last : "\$url"}}}])

Explain aggregation pipelines & Map reduce concept.

Ans: -

Aggregation pipelines give you a way to transfer and combined document in your collection.

It is very similar to UNIX pipeline command.

In the pipeline concept, you can send the output from one command to another as input and then go on.

The aggregate help to take an array of pipeline operator in the cell.

Mongodb also support the same concept in aggregation framework where a set of possible stage and set of documents used for producing a result and then the result will be found as JSON document at the end of application pipeline.

There are different possible stages in aggregation framework, which are as under.

- (1) \$ project - It is used to select some specific field from a collection.
- (2) \$ match - This is filtering operation for reduce the amount of document which will be given as input to the next stage.
- (3) \$group - It is used for simple count operations where the actual aggregation done.
- (4) \$sort – It is used to sort the documents.
- (5) \$skip – It is used to skip the documents from given amount of documents.

(6) \$ limit – It is used to limit the amount of documents by the given number starting from current position.

Map Reduce.

Ans: -

Map reduce is powerful and flexible tool for aggregating data.

It can be solving some problems that are too complex for aggregation framework.

Map reduce is two step approach to data processing it means first you have to map the data and then after reduce the data. In MongoDB we can use the map reduce command on a collection.

Map reduce use JavaScript as its query language so it can perform complex logics.

Map reduce is slow so it should not be used for real time data analysis.

Map reduce is generally used for processing the data in large volume of data sets.

Map reduce can work with multiple servers and splits a problem into chunks & send it to different machines, when all machine is finished its work then they merge all the pieces of the solution back into a full solution.

Map reduce is useful and powerful but it is complex.

The map reduce function first queries the collection then maps the result documents to emit key-value pairs which is then reduce based on the keys that have multiple value.

MapReduce Command

Following is the syntax of the basic mapReduce command –

```
>db.collection.mapReduce(  
function () {emit(key,value);}, //map function  
function(key,values) {return reduceFunction}, { //reduce function  
out: collection,  
query: document,  
sort: document,  
limit: number  
} )
```

The map-reduce function first queries the collection, then maps the result documents to emit key-value pairs which is then reduced based on the keys that have multiple values.

In the above syntax

□ **map** is a javascript function that maps a value with a key and emits a key-value pair

□ **reduce** is a javascript function that reduces or groups all the documents having the same key

- ❑ **out** specifies the location of the map-reduce query result
- ❑ **query** specifies the optional selection criteria for selecting documents
- ❑ **sort** specifies the optional sort criteria
- ❑ **limit** specifies the optional maximum number of documents to be returned

Using MapReduce

Consider the following document structure storing user posts. The document stores user_name of the user and the status of post.

```
{  
  "post_text": "tutorialspoint is an awesome website for tutorials",  
  "user_name": "mark",  
  "status": "active"  
}
```

Now, we will use a mapReduce function on our **posts** collection to select all the active posts, group them on the basis of user_name and then count the number of

posts by each user using the following code –

```
>db.posts.mapReduce  
  
(  
  function () {emit(this.user_id,1);},  
  function (key, values) {return Array.sum(values)},  
  {  
    query: {status:"active"},  
    out:"post_total"  
  }  
)
```

The above mapReduce query outputs the following result –

```
{  
  "result": "post_total",  
  "timeMillis": 9,  
  "counts": {  
    "input": 4,  
    "emit": 4,  
    "reduce": 2,  
    "output": 2  
  },  
}
```

```
"ok": 1,  
}
```

The result shows that a total of 4 documents matched the query (status:"active"), the map function emitted 4 documents with key-value pairs and finally the reduce function grouped mapped documents having the same keys into 2.

To see the result of this mapReduce query use the find operator –

```
>db.posts.mapReduce(  
  function () {emit(this.user_id,1);},  
  function (key, values) {return Array.sum(values)}, {  
    query: {status:"active"},  
    out:"post_total"  
  }  
). find ()
```

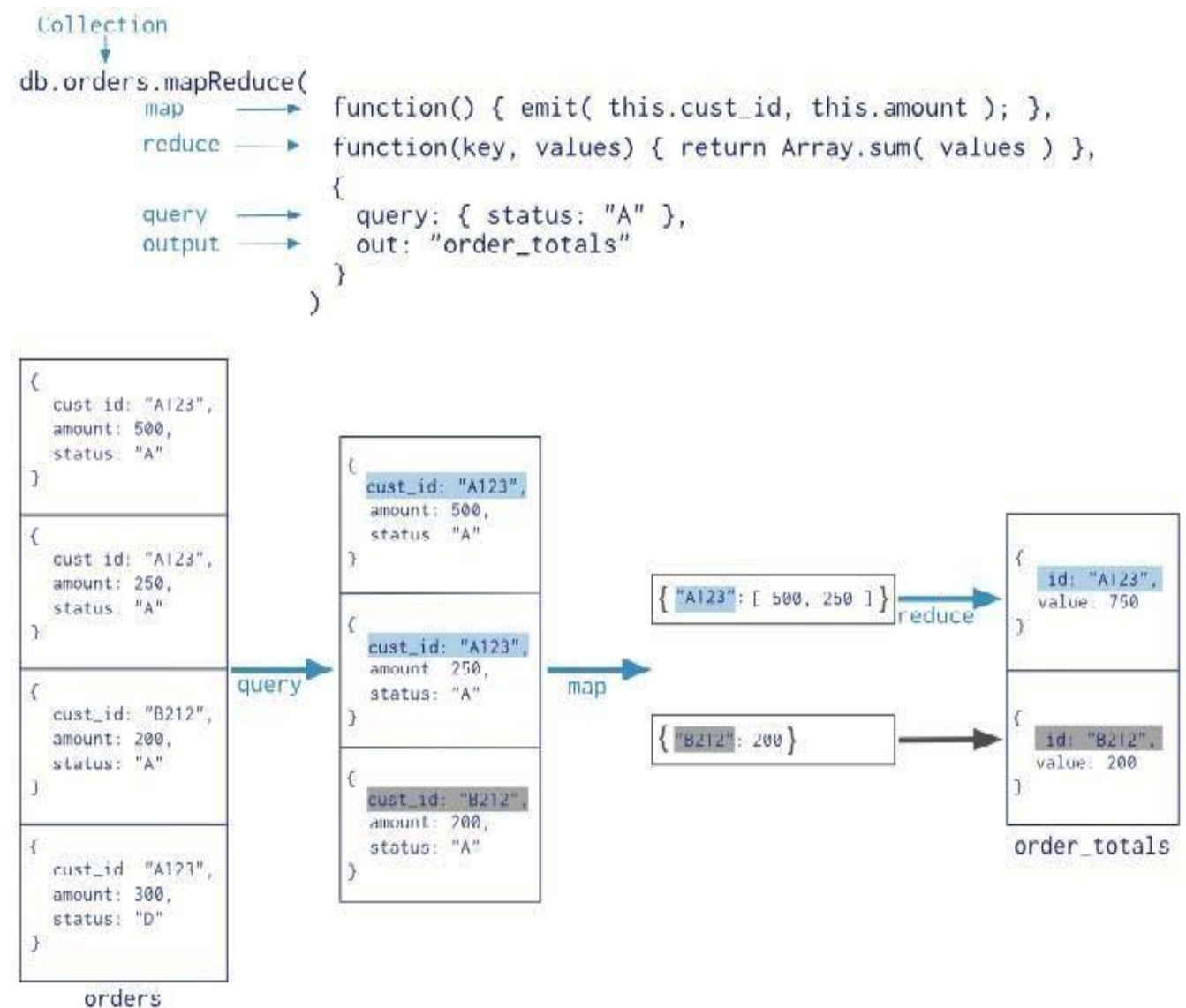
The above query gives the following result which indicates that both users **tom** and **mark** have two posts in active states –

```
{ "_id": "tom", "value": 2 }  
{ "_id": "mark", "value": 2 }
```

In similar manner, MapReduce queries can be used to construct large complex aggregation queries. The use of custom Javascript functions makes usage of MapReduce very flexible and powerful.

Map-reduce is a data processing paradigm for condensing large volumes of data into useful *aggregated* results. For map-reduce operations, MongoDB provides the [MapReduce](#) database command.

Consider the following map-reduce operation:



```
db.orders.aggregate(  
[  
  {$match: {status: "A"} },  
  {$group: {_id: "$cust_id", total: {$sum: "$amount"}}},  
  {$sort: {total: -1}}  
],  
{  
  explain: true  
}  
)
```

In this map-reduce operation, MongoDB applies the *map* phase to each input document (i.e., the documents in the collection that match the query condition). The map function emits key-value pairs. For those keys that have multiple values, MongoDB applies the *reduce* phase, which collects and condenses the aggregated data. MongoDB then stores the results in a collection. Optionally, the output of the reduce function may pass through a *finalize* function to further condense or process the results of the aggregation.

Example of Map Reduce

```
db.books.insert( {  
  
  "title": "MongoDB: The Definitive Guide",  
  
  "published": "2013-05-23",  
  
  "authors": [  
  
    { "firstName": "BRIJESH", "lastName": "PATEL" }  
  
  ],  
  
  "categories": [ "Databases", "NoSQL", "Programming" ],  
  
  "publisher": { "name": "NACHIKETA" },  
  
  "price": 32.99  
  
} )
```

```
db.books.insert( {  
  
  "title": "MongoDB in Action",  
  
  "published": "2011-12-16",  
  
  "authors": [  
  
    { "firstName": "JAYESH", "lastName": "PATEL" }  
  
  ],  
  
  "categories": [ "Databases", "NoSQL", "Programming" ],  
  
  "publisher": { "name": "JAMNADAS" },  
  
  "price": 30.83  
  
} )
```

Shree Matrumandir College Of IT & MGMT

```
db.books.insert( {  
  "title": "MongoDB in Action 2",  
  "published": "2011-12-16",  
  "authors": [  
    { "firstName": "JIGNESH", "lastName": "THANKI" }  
  ],  
  "categories": [ "Databases", "NoSQL", "Programming" ],  
  "publisher": { "name": "JAMNADAS" },  
  "price": 35.30  
} )
```

```
db.books.insert( {  
  "title": "Scaling MongoDB",  
  "published": "2011-03-07",  
  "authors": [  
    { "firstName": "NIKHIL", "lastName": "DAVE" }  
  ],  
  "categories": [ "Databases", "NoSQL" ],  
  "publisher": { "name": "NIRAV PRAKASHAN" },  
  "price": 25.30  
} )
```

COMMAND: -

```
db.runCommand( {  
  
  mapReduce: "books",  
  
  map: function () {  
  
    for (var index = 0; index < this.authors.length; ++index) {  
  
      var author = this.authors[ index ];  
  
      emit (author.firstName + " " + author.lastName, 1 );  
  
    }  
  
  },  
  
  reduce: function (author, counters) {  
  
    count = 0;  
  
    for (var index = 0; index < counters.length; ++index)  
  
    {  
  
      count += counters[index];  
  
    }  
  
    return count;  
  
  },  
  
  out: {inline: 1}  
  
} )
```

OUTPUT

```
{  
  "results": [  
    {  
      "_id": "BRIJESH PATEL",  
      "value": 1  
    },  
    {  
      "_id": "BRIJESH PATEL",  
      "value": 3  
    },  
    {  
      "_id": "NIKHIL DAVE",  
      "value": 2  
    },  
    {  
      "_id": "JIGNESH THANKI",  
      "value": 1  
    },  
    {  
      "_id": "JAYESH PATEL",
```

```
"value": 1
}
],
"counts": {
"input": 8,
"emit": 11,
"reduce": 2,
"output": 8
},
"ok": 1
}
```

create hns database and create website collection with 10 documents and perform different aggregate function on it.

Use hns

```
Db.website.insert({"id":1, "domain": "test1.com",
"hosting": "google.com"})
```

Shree Matrumandir College Of IT & MGMT

Id	Domain name	Hosting
1	Test1.com	www.google.com
2	Test2.com	www.amazon.com
3	Test3.com	www.amazon.com
4	Test4.com	www.rediff.com
5	Test5.com	www.w3school.com
6	Test6.com	www.google.com
7	Test7.com	www.amazon.com
8	Test8.com	www.google.com
9	Test9.com	www.rediff.com
10	Test10.com	www.w3school.com

(1) Group (): -

Db.website.aggregate(

{

\$group: {_id:" \$hosting", \$total: {\$sum:1}}


```
} );
```

(2) sorting (): -

```
Db.website.aggregate(  
  
{  
  
$group: {_id:" $hosting", total: {$sum:1}}  
  
},  
  
{  
  
$sort: {total: -1}  
  
}  
  
);
```

Where -1 is for descending and +1 is for ascending order.

(3) match ():-

```
Db.website.aggregate(  
  
{  
  
$match: {hosting: www.amazon.com}  
  
},  
  
{  
  
$ group: {_id:" $hosting", total: {$sum:1}}  
  
}  
  
);
```

(4) unwind (): -

It is used to unwind document data using array. It is helpful to separate the document.

Inventory document

Id 1

Item abc

Size S, M, L

```
Db.inventory.aggregate([ {unwind: "$size" }])
```

Output: -

```
{ "id": 1, "item": "abc", "size": "S" }
```

```
{ "id": 1, "item": "abc", "size": "M" }
```

```
{ "id": 1, "item": "abc", "size": "L" }
```

(5) skip (): -

Syntax: - { \$skip: <positive integer> }

```
Db.artical.aggggregate
```

```
{
```

```
Skip: 5
```

```
}
```

Output: - it will skip the first 5 documents passed to it by the pipeline.

(6) \$out (): -

It takes the document return by the pipeline and write them to the specified collection.

Syntax: - {\$out: "<output – collection>"}

This method creates a new collection in the database if it does not exist.

Example: -

Document book

Id	Title	Auth or	Copi es
1	Abc	X	2
2	Xyz	Y	2
3	Pqr	X	1
4	Xyz	Z	1

(7) \$project (): -

It is used to pass the specified field to the next stage in the pipeline.

Syntax: - {\$project: {<specifications>}}

Db.books.aggregate((\$project: {title:1, author:1}}))

Consider a book collection with the following document...

{

Id-1, title –abc123, isbn-0001122223333, author-last zzz first aaa, copies -3}

Output: -

```
{“id”:1, “title”:”abc123”, “author”: {“last”:”zzz”, “first”:”aaa”}}
```

(8) \$limit (): -

It is used to limit the no of documents passed to the next stage in the pipeline.

The \$limit stage has the following prototype form:

```
{ $limit: <positive integer> }
```

\$limit takes a positive integer that specifies the maximum number of documents to pass along.

Example

```
db.article.aggregate({ $limit : 5 } );
```

This operation returns only the first 5 documents passed to it from by the pipeline.

\$limit has no effect on the content of the documents it passes.

(9) count()

db.collection.count(*query, options*)

Returns the count of documents that would match a [find\(\)](#) query. The db.collection.count() method does not perform the [find\(\)](#) operation but instead counts and returns the number of results that match a query.

Parameter	Type	Description
query	document	The query selection criteria.
options	document	Optional. Extra options for modifying the count.

The options document contains the following fields:

Field	Type	Description
limit	Integer	Optional. The maximum number of documents to count.
skip	Integer	Optional. The number of documents to skip before counting.
hint	string or document	Optional. An index name hint or specification for the query.

To count the number of all documents in the orders collection, use the following operation:

```
db.orders.count()
```

This operation is equivalent to the following:

```
db.orders.find().count()
```

output:- 4

Count all Documents that Match a Query

Count the number of the documents in the orders collection with the field ord_dt greater than new Date('01/01/2012'):

```
db.orders.count( { ord_dt: { $gt: new Date('01/01/2012') } } )
```

The query is equivalent to the following:

```
db.orders.find( { ord_dt: { $gt: new Date('01/01/2012') } } ).count()
```

(10) Distinct()

```
db.collection.distinct(field, query)
```

Finds the distinct values for a specified field across a single collection and returns the results in an array.

Parameter	Type	Description
field	string	The field for which to return distinct values.
query	document	A query that specifies the documents from which to retrieve the distinct values.

Shree Matrumandir College Of IT & MGMT

The `db.collection.distinct()` method provides a wrapper around the [distinct](#) command. Results must not be larger than the maximum [BSON size](#).

Array Fields

If the value of the specified field is an array, `db.collection.distinct()` considers each element of the array as a separate value.

For instance, if a field has as its value `[ 1, [1], 1 ]`, then `db.collection.distinct()` considers 1, `[1]`, and 1 as separate values.

For an example, see [Return Distinct Values for an Array Field](#).

Index Use

When possible, `db.collection.distinct()` operations can use indexes.

Indexes can also [cover](#) `db.collection.distinct()` operations.

Examples

The examples use the inventory collection that contains the following documents:

```
{ "_id": 1, "dept": "A", "item": { "sku": "111", "color": "red" }, "sizes": [ "S",  
"M" ] }
```

```
{ "_id": 2, "dept": "A", "item": { "sku": "111", "color": "blue" }, "sizes": [ "M",  
"L" ] }
```

```
{ "_id": 3, "dept": "B", "item": { "sku": "222", "color": "blue" }, "sizes": "S" }
```

```
{ "_id": 4, "dept": "A", "item": { "sku": "333", "color": "black" }, "sizes": [ "S" ]  
}
```


Return Distinct Values for a Field

The following example returns the distinct values for the field dept from all documents in the inventory collection:

```
db.inventory.distinct( "dept" )
```

The method returns the following array of distinct dept values:

```
[ "A", "B" ]
```

Return Distinct Values for an Embedded Field

The following example returns the distinct values for the field sku, embedded in the item field, from all documents in the inventory collection:

```
db.inventory.distinct( "item.sku" )
```

The method returns the following array of distinct sku values:

```
[ "111", "222", "333" ]
```

Return Distinct Values for an Array Field

The following example returns the distinct values for the field sizes from all documents in the inventory collection:

```
db.inventory.distinct( "sizes" )
```

The method returns the following array of distinct sizes values:

```
[ "M", "S", "L" ]
```

Specify Query with distinct

The following example returns the distinct values for the field sku, embedded in the item field, from the documents whose dept is equal to "A":

```
db.inventory.distinct( "item.sku", { dept: "A" } )
```

The method returns the following array of distinct sku values:

```
[ "111", "333" ]
```